

Dokumentacja Projektowa: Królestwo Krasnoludków (ASD 2)

Projekt realizowany w ramach przedmiotu Algorytmy i Struktury Danych 2. System rozwiązuje złożone problemy logistyczno-geometryczne, wykorzystując autorski silnik w C++ oraz interfejs graficzny w Pythonie (PySide6).

1 Problem 1: Przydział krasnoludków do kopalni (Minimalizacja Owsianki)

1.1 Sformalizowanie problemu

Zadanie polega na znalezieniu optymalnego skojarzenia w grafie dwudzielnym określającym preferencje. Jednym zbiorem wierzchołków są obiekty klasy `Krasnoludek` (liczba N), a drugim obiekty klasy `Kopalnia` (liczba M). Każda kopalnia posiada ograniczoną pojemność C (określoną jako `iloscMiejsc`), a każdy krasnoludek może pracować tylko w kopalni wydobywającej jeden z akceptowanych przez niego surowców (zdefiniowanych w wektorze `mineraly`). Celem jest znalezienie takiego przydziału (maksymalizującego sumaryczną liczbę pracujących krasnoludków), który jednocześnie minimalizuje sumaryczną odległość euklidesową (koszt transportu/owsianki) między współrzędnymi domków (`domek.x`, `domek.y`) a wyrobiskami (`wspolrzedne.x`, `wspolrzedne.y`).

1.2 Wykorzystane struktury danych

Problem został zamodelowany za pomocą sieci przepływowej w ramach klasy `Graph`.

- **Wierzchołki:** W grafie zdefiniowano $N + M + 2$ wierzchołki: krasnoludki (indeksowane od 1 do N), kopalnie (indeksowane od $N + 1$ do $N + M$), sztuczne źródło `zrodlo` oraz sztuczne ujście `ujscie`.
- **Krawędzie:** Zaimplementowane jako niestandardowa struktura `Edge` przechowująca: wierzchołek docelowy (`to`), maksymalną przepustowość (`capacity`), aktualny przepływ (`flow`), odległość jako koszt (`cost`) oraz indeks krawędzi powrotnej (`rev_idx`). Indeks powrotny umożliwia w czasie stałym odwołanie się do przeciwnej krawędzi w grafie rezydualnym.
- **Reprezentacja grafu:** Wykorzystano listy sąsiedztwa w postaci wektora wektorów `std::vector<std::vector<Edge>> adj`, co zapewnia sprawną iterację po krawędziach wychodzących z danego wierzchołka.

1.3 Opis algorytmu (Cycle Canceling / Min-Cost Max-Flow)

Rozwiązanie zaimplementowano w oparciu o algorytm Min-Cost Max-Flow w wariantcie Cycle Canceling (niwelowanie cykli). Zostało ono ustrukturyzowane w metodach klasy Graph:

- **Inicjalizacja i budowa grafu (buildGraph):** Tworzone są krawędzie ze źródła do każdego krasnoludka (przepustowość 1, koszt 0). Następnie sprawdzane są preferencje – jeśli surowiec kopalni znajduje się na liście minerałów krasnoludka, tworzona jest krawędź o przepustowości 1, której kosztem jest zaokrąglona odległość euklidesowa obliczana funkcją `round(hypot(dx, dy))`. Kopalnie są łączone z ujściem krawędziami o przepustowości równej `iloscMiejsc` i koszcie 0.
- **Faza 1 (Algorytm Edmondsa-Karpa - findMaxFlow):** Służy znalezieniu początkowego, maksymalnego przepływu, ignorując koszty. Wykorzystano w niej przeszukiwanie wszerz (`std::queue<int> q`), które iteracyjnie wyszukuje ścieżki powiększające w grafie i aktualizuje stany przepływu na krawędziach bazowych oraz powrotnych. Wartość zwrotna to całkowita liczba zatrudnionych krasnoludków.
- **Faza 2 (Bellman-Ford i niwelowanie cykli - minCostMaxFlow):** Minimalizuje koszty przepływu. Na utworzonym w Fazie 1 grafie rezydualnym iteracyjnie wyszukiwane są cykle o ujemnym koszcie. Do ich detekcji dokonuje się relaksacji krawędzi, sprawdzając warunek `e.capacity > e.flow && dist[e.to] > dist[u] + e.cost`. Po wykryciu wierzchołka uwikłanego w cykl (`node_in_cycle`), cykl jest rekonstruowany za pomocą wektorów `parent` i `edge_to_parent`, a następnie przesyłany jest przez niego przepływ (równy minimalnej dostępnej przepustowości wzdłuż cyklu), co globalnie zmniejsza dystans pokonywany przez pracowników.
- **Ekstrakcja wyników (przypiszWyniki):** Metoda iteruje po grafie z rozwiązaniem przepływem, i dla każdego krasnoludka identyfikuje krawędź o dodatnim przepływie skierowaną do kopalni, odpowiednio przypisując `ID_kopalni` do struktury danego pracownika.

1.4 Analiza poprawności

- Utworzenie krawędzi ze źródła do każdego krasnoludka o maksymalnej przepustowości równej 1 i weryfikacja jej podczas pchania przepływu (`e.capacity > e.flow`) gwarantuje, że pojedynczy krasnoludek zostanie przydzielony maksymalnie do jednego miejsca pracy.
- Analogiczne ograniczenie krawędzi od kopalni do ujścia warunkuje nierzekraczalność pojemności wyrobisk.
- Twierdzenie Kleina o niwelowaniu cykli dowodzi, że przepływ ma minimalny koszt wtedy i tylko wtedy, gdy w jego sieci rezydualnej nie ma skierowanych cykli o ujemnym koszcie. Pętla w fazie drugiej rygorystycznie wyszukuje ujemne cykle, dopóki zmienna `node_in_cycle` nie pozostanie na wartości `-1`, co matematycznie i logicznie zapewnia uzyskanie optymalnego (minimalnego) dystansu dla zadanej liczby przypisań.

1.5 Złożoność

- **Złożoność pamięciowa:** $\mathcal{O}(V + E)$, gdzie $V = N + M + 2$, a E to liczba dopuszczalnych przypisań krasnoludków do kopalni. Pamięć rośnie liniowo dzięki wykorzystaniu dynamicznych struktur `std::vector` do tworzenia list sąsiedztwa i wektorów śledzących. Kluczową optymalizacją pamięciową wprowadzającą zyski wydajnościowe jest alokacja wektorów pomocniczych (`dist`, `parent`, `edge_to_parent`) przed główną pętlą `while` i użycie na nich funkcji `std::fill` w każdym cyklu, co eliminuje ciągłe tworzenie i usuwanie struktur przy każdej iteracji.
- **Złożoność czasowa:** Faza 1 oparta na procedurze Edmondsa-Karpa działa w granicznym czasie $\mathcal{O}(V \cdot E^2)$. W Fазie 2 pojedyncze wywołanie algorytmu Bellmana-Forda wymaga czasu $\mathcal{O}(V \cdot E)$. Metoda Cycle Canceling w wariancie pesymistycznym jest zależna od maksymalnego sumarycznego kosztu w sieci, co sprawia że liczba iteracji może być proporcjonalna do maksymalnej owsianki, niemniej przy relatywnie małych, naturalnych odległościach pomiędzy kopalniami a domkami algorytm jest niezwykle skuteczny operacyjnie. Właściwa implementacja tablic i przejść gwarantuje, że proces konverguje sprawnie do wartości optymalnej w środowisku produkcyjnym.

2 Problem 2: Trasa Patrolowa Księcia (Obrona przed Jabłkami)

2.1 Sformalizowanie problemu

Zadanie polega na wyznaczeniu najkrótszej zamkniętej ścieżki (wielokąta o minimalnym obwodzie), która otacza wszystkie punkty reprezentujące aktywne kopalnie w przestrzeni dwuwymiarowej $\mathbb{R}^2$. Aktywnymi kopalniami są te wyrobiska, do których w ramach optymalizacji przepływów w Problemie 1 zostali pomyślnie przydzieleni pracownicy. W ujęciu matematycznym i geometrycznym problem ten sprowadza się do znalezienia **otoczki wypukłej (Convex Hull)** dla wyznaczonego skończonego zbioru punktów P , a następnie obliczenia jej obwodu, co definiuje optymalną, zgeometryzowaną trasę patrolową zabezpieczającą granice ekonomiczne królestwa.

2.2 Wykorzystane struktury danych

Implementacja opiera się na niskopoziomowych, wydajnych strukturach danych, które zapewniają minimalny narzut pamięciowy i bezpośrednią integrację z modułem grafowym:

- **Współrzędne (Wspolrzedne):** Struktura przechowująca współrzędne kartezjańskie jako liczby całkowite typu `int` (`x` oraz `y`), co zapobiega powstawaniu błędów zaokrągleń oraz problemów z precyzją maszynową podczas reprezentacji położenia obiektów na siatce geograficznej.
- **Kolekcja punktów i stos:** Kontener dynamiczny `std::vector<Wspolrzedne>` służy zarówno jako pierwotny zbiór wejściowy aktywnych kopalni poddawany procedurze sortowania, jak i efektywna implementacja stosu (za pomocą metod `push_back` oraz `pop_back`) przechowującego sekwencję wierzchołków aktualnie tworzących konstruowaną otoczkę wypukłą.

- **Integracja z siecią przepływową:** Przed uruchomieniem algorytmu geometrycznego następuje filtracja obiektów. Graf rezydualny z klasy `Graph` poddawany jest analizie strukturalnej. Dana kopalnia jest uznawana za aktywną i kwalifikowana do zbioru P wtedy i tylko wtedy, gdy krawędź incydentna łącząca wierzchołek tej kopalni z super-ujściem (`ujscie`) wykazuje dodatni przepływ (`e.flow > 0`), co oznacza faktyczne obsadzenie stanowisk wydobywczych przez co najmniej jednego krasnoludka.

2.3 Opis algorytmu (Algorytm Grahama)

Do wyznaczenia otoczki wypukłej zaimplementowano deterministyczny **Algorytm Grahama (Graham's Scan)**. Proces ten przebiega w czterech kluczowych fazach:

1. **Wybór punktu startowego (P0):** Wyszukiwany jest wierzchołek o globalnie minimalnej współrzędnej y . W przypadku wystąpienia remisu (współliniowość pozioma), kryterium rozstrzygającym jest najmniejsza wartość współrzędnej x . Punkt ten staje się początkiem układu bieguna odniesienia i ma gwarancję przynależności do otoczki.
2. **Sortowanie kątowe:** Wszystkie pozostałe punkty zbioru są sortowane rosnąco według kąta biegunowego, jaki tworzą z punktem P0. W celu maksymalizacji wydajności obliczeniowej oraz uniknięcia kosztownych operacji trygonometrycznych (takich jak funkcja `atan2`) i błędów precyzji zmiennoprzecinkowej, relację porządku zaimplementowano przy użyciu predykatu geometrycznego opartego na **iloczynie wektorowym** (dedykowana funkcja `skret`).
3. **Konstrukcja otoczki (Skanowanie):** Algorytm sekwencyjnie przetwarza posortowaną listę punktów. Przechodząc do kolejnego punktu, badana jest orientacja trzech ostatnich wierzchołków (dwóch ze stosu oraz nowego punktu kandydackiego). Wykorzystując funkcję obliczającą wyznacznik macierzy 2×2 (iloczyn wektorowy), sprawdzany jest kierunek zwrotu wektorów. Jeśli punkty wykonują "skręt w prawo" lub są współliniowe, oznacza to naruszenie wypukłości (wkłęsłość struktury). Wówczas ostatni punkt ze stosu jest usuwany (`pop_back`). Procedura ta powtarza się wstecznie, aż do momentu uzyskania ścisłego "skrętu w lewo", po czym nowy punkt jest odkładany na stos.
4. **Kalkulacja obwodu:** Po zamknięciu cyklu i powrocie do punktu startowego, następuje obliczenie ostatecznej długości trasy. Wykorzystywana jest do tego funkcja `std::hypot` z biblioteki `<cmath>`, która w sposób bezpieczny numerycznie (zabezpieczając przed niedomiarami i nadmiarami przy potęgowaniu dużych liczb całkowitych) wyznacza odległości euklidesowe pomiędzy sąsiednimi wierzchołkami wielokąta zapisanego na stosie, a ich suma stanowi końcowy wynik.

2.4 Analiza poprawności

Algorytm Grahama poprawnie wyznacza otoczkę wypukłą, co wynika bezpośrednio z niezmiennika pętli skanującej oraz matematycznych własności iloczynu wektorowego. Znak wyznacznika par wektorów zaczepionych w analizowanych punktach jednoznacznie i błędnie wskazuje orientację przestrzenną (dodatni dla skrętu w lewo, ujemny dla skrętu w prawo, zero dla punktów współliniowych).

- **Obsługa przypadków brzegowych (Współliniowość):** Punkty leżące na tej samej prostej przechodzącej przez P_0 są odpowiednio filtrowane lub porządkowane za pomocą dodatkowego kryterium odległościowego – w przypadku identycznego kąta biegunowego, priorytet ma punkt położony najdalej, co realizowane jest za pomocą porównywania kwadratów odległości euklidesowych (brak pierwiastkowania chroni przed utratą dokładności).
- **Niewielka liczba punktów aktywnych:** Jeśli liczba aktywnych kopalni $K < 3$, algorytm wykonuje bezpieczne przejście awaryjne (early exit):
 - Dla $K = 0$ lub $K = 1$ zwracana jest długość trasy równa 0.
 - Dla $K = 2$ trasa patrolowa degeneruje się do odcinka łączącego te dwie kopalnie, a algorytm poprawnie wyznacza podwojoną odległość euklidesową pomiędzy nimi (reprezentującą drogę patrolu „tam i z powrotem”).

2.5 Złożoność

Niech K oznacza liczbę aktywnych kopalni wyselekcjonowanych na podstawie wyników sieci przepływowej ($K \leq M$).

- **Złożoność pamięciowa:** $\mathcal{O}(K)$ – zapotrzebowanie na pamięć jest optymalne i ściśle liniowe względem liczby przetwarzanych punktów. Algorytm operuje na wektorach zaalokowanych na stosie lub jako ciągłe bloki pamięci dynamicznej, nie tworząc żadnych rozproszonych struktur wskaźnikowych czy drzewiastych, co gwarantuje doskonałą lokalność danych w pamięci podręcznej procesora (cache locality).
- **Złożoność czasowa:** $\mathcal{O}(K \log K)$ – całkowity czas wykonania determinowany jest przez etap sortowania kąтового, realizowany za pomocą wywołania szablonu `std::sort`, którego asymptotyczny koszt wynosi $\mathcal{O}(K \log K)$. Samo skanowanie Grahama (faza usuwania wklęsłości) działa w czasie liniowym $\mathcal{O}(K)$, ponieważ każdy punkt jest odkładany na stos dokładnie raz i może zostać z niego zrzuty maksymalnie raz. Wstępne wyłowienie aktywnych kopalni poprzez iterację po krawędziach ujścia grafu przepływowego zajmuje czas $\mathcal{O}(M)$, co przy uwzględnieniu relacji $K \leq M$ nie wpływa negatywnie na ostateczną złożoność obliczeniową algorytmu geometrycznego.

3 Problem 3: Salwa Dekametrowców (Obrona Przeciwlotnicza)

3.1 Sformalizowanie problemu

Problem polega na błyskawicznym wyłonieniu dowódcy obrony (dekametrowca o największej "głośności") na konkretnym, zaatakowanym odcinku granicy. Granicę państwa reprezentuje zbiór N dekametrowców rozstawionych wzdłuż otoczki wypukłej wyznaczonej w Problemie 2. System musi sprawnie odpowiadać na zapytania o maksimum w zadanym przedziale indeksów $[L, R]$. Ponieważ granica jest krzywą zamkniętą (pętlą), ataki mogą "przechodzić" przez punkt zszycia tablicy, co oznacza, że indeks początkowy w logice nadrzędnej może być matematycznie większy od końcowego ($L > R$).

3.2 Wykorzystane struktury danych

Dla uzyskania logarytmicznego czasu odpowiedzi wykorzystano strukturę **Drzewa Przedziałowego (Segment Tree)** zaimplementowaną na statycznej tablicy wewnątrz klasy `DrzewoPrzedzialowe`.

- **Reprezentacja drzewa:** Wektor `std::vector<WezelDrzewa> tree` o rozmiarze $4N+1$, gdzie każdy węzeł przechowuje strukturę z polami `maxGlosnosc` oraz `straznikID`.
- **Element neutralny:** Do obsługi pustych wywołań (poza zakresem) zdefiniowano węzeł `ELEMENT_NEUTRALNY`, w którym głośność to stała `MINUS_NIESKONCZONOSC` (równa -1000000000), a ID strażnika wynosi -1 . Gwarantuje to, że zapytania wychodzące poza zakres nie zaburzają wyników funkcji zwracających maksimum.
- **Dane wejściowe:** Wektor bazowy `std::vector<Dekametrowiec> A`, generowany dynamicznie i przypisywany w konstruktorze. Struktura `Dekametrowiec` przechowuje ID oraz `glosnosc`.

3.3 Opis algorytmu (Range Maximum Query - RMQ)

Działanie modułu opiera się na klasycznym rozwiązaniu problemu RMQ, urozmaiconym o deterministyczne reguły rozstrzygania remisów:

- **Budowa drzewa (Funkcja BUILD):** Konstrukcja przebiega w sposób rekurencyjny (Top-Down). Przestrzeń tablicy dzielona jest na połowy aż do osiągnięcia liści ($1 == r$), do których wpisywane są początkowe głośności oraz identyfikatory dekametrowców z wektora bazowego. Przy powrocie z rekurencji, wartość węzła wewnętrznego v jest wyznaczana na podstawie dwojga dzieci: lewego $2v$ i prawego $2v+1$. Jeśli głośności dzieci są równe, algorytm w ramach funkcji `f` wybiera strażnika o mniejszym ID (`L.straznikID <= R.straznikID ? L : R`), co zapewnia w pełni deterministyczne zachowanie systemu przy remisach.
- **Przeszukiwanie (Funkcja QUERY):** Aby znaleźć maksimum na przedziale $[L, R]$, drzewo jest przeszukiwane rekurencyjnie.
 1. Jeśli rozpatrywany węzeł jest całkowicie rozłączny z zadanym zapytaniem ($r < ql \ || \ qr < l$), algorytm natychmiast zwraca `ELEMENT_NEUTRALNY`.
 2. Jeśli węzeł w całości zawiera się w przedziale zapytania ($ql <= l \ \&\& \ r <= qr$), zwracana jest jego zawartość wprost (`tree[v]`) bez dalszego schodzenia w dół, co gwarantuje logarytmiczną wydajność.
 3. W przypadku częściowego pokrycia, przedział jest dzielony przez środek ($mid = (l + r) / 2$), a wynik to skalkulowane maksimum z lewego i prawego poddrzewa (z analogicznym deterministycznym uwzględnieniem identyfikatorów ID).
- **Zapytania cykliczne (Atak dookoła granicy):** Ze względu na wbudowane zabezpieczenie w metodzie `zapytajONajglosniejszego`, która zwraca -1 w przypadku podania błędnego przedziału (gdzie $ql > qr$), nadrzędna logika systemu jest odpowiedzialna za obsługę zapytań cyklicznych. Jeżeli $L \leq R$, wystarcza jedno wywołanie struktury. Jeżeli $L > R$, problem musi być rozbity na dwa wywołania: na

przedział od L do końca tablicy $(N-1)$ oraz od początku tablicy (0) do R . Ostatecznym dowódcą zostaje krasnoludek wyłoniony z porównania maksimum tych dwóch niezależnych wyników.

3.4 Analiza poprawności

Poprawność drzewa przedziałowego opiera się na własności łączności funkcji statystycznej – funkcja wyznaczania maksimum jest w pełni łączna, tj. $\max(a, b, c) = \max(\max(a, b), c)$. Rozbicie przedziału w wewnętrznych węzłach drzewa pozwala na bezbłędną agregację danych. Zaimplementowana w metodach BUILD i QUERY rozszerzona logika selekcji przy remisach gwarantuje determinizm, niwelując ryzyko losowych zachowań systemu w przypadku wielu dekametrowców o tej samej, najwyższej głośności. Logika rozbijania zapytań cyklicznych również jest matematycznie poprawna, opierając się na tożsamości $\max(A \cup B) = \max(\max(A), \max(B))$, co idealnie modeluje domkniętą granicę.

3.5 Złożoność

Niech N oznacza liczbę dekametrowców na granicy (liczbę węzłów bazowych / liści struktury).

- **Złożoność pamięciowa:** $\mathcal{O}(N)$ – reprezentacja tablicowa pełnego drzewa binarnego wymaga rezerwacji ciągłego bloku pamięci dla wektora `tree` o rozmiarze rzędu $4N + 1$ elementów, co omija potrzebę wskaźników i gwarantuje świetną lokalność cache (pamięć podręczna).
- **Złożoność czasowa budowy:** $\mathcal{O}(N)$ – każdy z około $2N$ węzłów drzewa jest odwiedzany i inicjalizowany dokładnie raz podczas pojedynczego wywołania rekurencyjnego funkcji BUILD.
- **Złożoność czasowa zapytania:** $\mathcal{O}(\log N)$ – w najgorszym scenariuszu wysokość drzewa to $\lceil \log_2 N \rceil$. Dzięki pre-agregacji danych w węzłach, maksymalna liczba odwiedzonych wierzchołków na każdym poziomie drzewa podczas wykonania procedury QUERY wynosi 4. W przypadku zapytania rozbitego (cyklicznego) wykonywane są de facto dwa takie zapytania, co asymptotycznie redukuje się i zachowuje stabilny rygor złożoności $2 \times \mathcal{O}(\log N) = \mathcal{O}(\log N)$.

4 Problem 4: Elektroniczne Księgi (Kompresja i Wyszukiwanie)

4.1 Sformalizowanie problemu

Zadanie polega na optymalizacji przechowywania obszernych archiwów królestwa (zmniejszenie objętości danych tekstowych – "elektronicznego papieru") oraz umożliwieniu błyskawicznego odnajdywania konkretnych fraz i słów w tekstach ksiąg. Problem algorytmiczny sprowadza się do implementacji bezstratnej kompresji tekstu na poziomie bitowym przy użyciu algorytmu Huffmana oraz szybkiego algorytmu wyszukiwania podciągów w długich łańcuchach znaków przy wykorzystaniu algorytmu Rabina-Karpa z zastosowaniem funkcji z przesuwającym się oknem (rolling hash).

4.2 Wykorzystane struktury danych

Dla realizacji tego problemu zastosowano i zaimplementowano w klasie `ElektroniczneKsiegi` dwie odrębne rodziny struktur:

- **Kompresja:**

- **Drzewo binarne:** Reprezentujące Drzewo Huffmana, zbudowane z dynamicznie alokowanych węzłów struktury `HuffmanNode`, przechowujących znak (`ch`), częstotliwość (`freq`) oraz wskaźniki na lewe i prawe poddrzewo.
- **Kolejka priorytetowa:** Szablon `std::priority_queue` typu Min-Heap, wykorzystujący niestandardowy komparator `CompareNodes` (sortujący według najmniejszej częstotliwości: `l->freq > r->freq`), co optymalizuje proces budowy drzewa.
- **Tablice asocjacyjne:** Słowniki `std::map` służące z jednej strony do zliczania częstotliwości wystąpień znaków przed budową drzewa, a z drugiej do trwałego mapowania znaków na unikalne kody prefiksowe (`std::map<char, std::string> huffmanCodes`).

- **Wyszukiwanie:**

- **Struktury wbudowane języka:** Łańcuchy znaków (`std::string`) oraz wektor `std::vector<int>` służący do kolekcjonowania wszystkich indeksów startowych odnalezionego wzorca.
- **Rejestry arytmetyczne:** Zmienne całkowitoliczbowe służące do przechowywania wartości funkcji haszujących, wykorzystujące operacje matematyczne modulo dla zabezpieczenia przed przepełnieniem (overflow).

4.3 Opis algorytmu (Huffman i Rabin-Karp)

Moduł został podzielony na dwa współpracujące, lecz niezależne algorytmy tekstowe:

- **Oszczędzanie papieru (Algorytm Huffmana):**

1. Pierwszym krokiem jest budowa słownika częstotliwości wszystkich unikalnych znaków występujących w ciągu wejściowym (`freq[ch]++`).
2. Następuje inicjalizacja kolejki priorytetowej, w której każdy unikalny znak ze swoim licznikiem staje się pojedynczym liściem-węzłem (`new HuffmanNode(pair.first, pair.second)`).
3. Algorytm iteracyjnie pobiera z kolejki dwa węzły o najmniejszej częstotliwości i łączy je w nowy węzeł nadrzędny o znaku pustym `\0`, sumując ich częstotliwości. Proces powtarza się aż do osiągnięcia przez kolejkę rozmiaru 1, co oznacza wyłonienie korzenia (`root`) całego drzewa.
4. Następnie uruchamiana jest rekurencyjna funkcja `generateCodes`, która przechodząc głęboko przez drzewo, buduje ciągi zer i jedynek (dołączając "0" przy zejściu w lewo i "1" przy zejściu w prawo), zapisując wynik w strukturze `huffmanCodes`.

5. Słownik oraz ciąg bitów zapisywane są bezpośrednio do strumienia plikowego przez funkcję `zapiszArchiwumNaDysk`, gwarantując spójność (rozmiar słownika na górze pliku ułatwia późniejsze wczytywanie).

- **Szybkie wyszukiwanie (Algorytm Rabina-Karpa):**

1. Zdefiniowano wielkość alfabetu ASCII jako stałą mnożnikową $d = 256$ oraz wprowadzono małą liczbę pierwszą $q = 101$ służącą do operacji modulo (%) w celu zabezpieczenia operacji przed wyjściem poza dopuszczalny zakres zmiennych.
2. Na starcie obliczany jest unikalny skrót matematyczny (hasz) p dla szukanego wzorca oraz dla początkowego "okna" tekstu t o tej samej długości.
3. Podczas przesuwania okna wzdłuż tekstu w pętli (`for (int i = 0; i <= n - m; i++)`), nowy hasz aktualizowany jest w czasie stałym. Realizowane jest to poprzez zniwelowanie wartości znaku wypadającego z lewej strony oraz dodanie wartości nowego znaku napotkanego po prawej stronie: $t = (d \cdot (t - \text{tekst}[i] \cdot h) + \text{tekst}[i + m]) \bmod q$.
4. Zabezpieczono potencjalne wystąpienie arytmetyki ujemnej poprzez wymuszenie normalizacji: `if (t < 0) t = (t + q)`.
5. Tylko w przypadku potencjalnej zgodności ($p == t$), algorytm iteracyjnie weryfikuje faktyczne podobieństwo znak-po-znak (`tekst[i + j] != wzorzec[j]`), by po pozytywnym teście zapisać indeks wystąpienia do wektora wynikowego.

4.4 Analiza poprawności

- **Algorytm Huffmana:** Zastosowanie budowy top-down gwarantuje stworzenie kodów prefiksowych – ze względu na to, że znaki znajdują się wyłącznie w liściach drzewa, kod reprezentujący określoną literę nigdy nie stanowi początku (prefiksu) innej litery. Rozwiązanie zastosowane w procedurze `dekompresuj` korzysta z `odwroconaMapa` i bezstratnie rekonstruuje dane buforując znaki, co bezbłędnie przydziela odczytane bity do oryginalnych symboli alfanumerycznych.
- **Algorytm Rabina-Karpa:** Użycie manualnego sprawdzenia sekwencji znaków w momencie kolizji wartości modulo jest rygorystyczne. Gwarantuje to brak wyników fałszywie pozytywnych (False Positives), które mogłyby pojawić się na skutek powtarzalności reszt z dzielenia dla różnych, lecz matematycznie symetrycznych układów ASCII. Wykorzystanie pierwszoliczbowej wartości $q = 101$ odpowiednio rozprasza hasze, zmniejszając prawdopodobieństwo wystąpienia sztucznej zgodności.

4.5 Złożoność

Niech N oznacza długość pełnego tekstu w księdze, M długość szukanego wzorca, a A wielkość alfabetu.

- **Złożoność pamięciowa:** Drzewo Huffmana oraz pomocnicze mapy dla znaków ASCII wymagają rozmiaru $\mathcal{O}(A)$. Z uwagi na to, że użyto standardowych typów danych języka C++ (maksymalnie $A = 256$), wielkość dodatkowych struktur rośnie do sztywnego poziomu asymptotycznego ograniczonego z góry, co sprowadza koszt

pamięciowy algorytmu kompresji do stałego $\mathcal{O}(1)$. Algorytm wyszukiwania działa w przestrzeni in-place, konsumując marginalnie $\mathcal{O}(1)$ na trzymanie wartości zmiennych z rejestrem wyników (wektorem wielkości proporcjonalnej do ilości dopasowań).

- **Złożoność czasowa kompresji (Huffman):** Obliczenie początkowej częstotliwości realizowane jest jednorazowym przejściem pętli $\mathcal{O}(N)$. Budowa struktury i ekstrakcja z Min-Heap to w najgorszym scenariuszu czas $\mathcal{O}(A \log A)$. Sumarycznie, proces kodowania to $\mathcal{O}(N + A \log A)$, co stanowi wysoce optymalne podejście deterministyczne.
- **Złożoność czasowa wyszukiwania (Rabin-Karp):** Etap przygotowania funkcji wielomianowej zajmuje $\mathcal{O}(M)$. Dalsza analiza "Rolling Hash" trwa asymptotycznie $\mathcal{O}(1)$ na każdą iterację przesuwanego okna (bez względu na długość ciągu). Oznacza to, że czas trwania wyniesie optymalne $\mathcal{O}(N + M)$, chyba że dojdzie do drastycznego i złośliwego występowania kolizji. W układzie skrajnie pesymistycznym, gdy dla wszystkich okien hasze będą sztucznie tożsame, czas ten zdegradowałby się do $\mathcal{O}(N \cdot M)$.

5 Metodologia Testowania i Analiza Wydajności

5.1 Środowisko testowe i automatyzacja badawcza

Proces weryfikacji poprawności algorytmów oraz dogłębnej analizy ich wydajności został zautomatyzowany przy użyciu skryptów napisanych w języku Python.

- **Generatory danych:** Do tworzenia zróżnicowanych zestawów wejściowych wykorzystano dedykowane skrypty `gen_krasnoludki.py` oraz `generuj_testy.py`. Narzędzia te umożliwiają symulowanie zarówno małych, kłopotliwych geometrycznie przypadków ułatwiających debugowanie, jak i potężnych populacji testowych (np. archiwa "mamut" i "duże"). Generatory dynamicznie losują współrzędne domków krasnoludków na dwuwymiarowej siatce, przydzielają preferowane przez nich surowce w oparciu o ustalone wagi prawdopodobieństwa, a następnie masowo eksportują gotowe układy do plików konfiguracyjnych w formacie `.csv` oraz natywnym binarnym.
- **Moduł benchmarkingu:** Orkiestrację testów obciążeniowych zapewnia skrypt profilujący `benchmark.py`. Odpowiada on za zautomatyzowaną kompilację, a następnie wielokrotne, sekwencyjne uruchamianie silnika C++ dla coraz większych przestrzeni danych. Po wykonaniu przebiegów, skrypt zbiera odczyty precyzyjnych timerów z poszczególnych modułów i agreguje zebrane wyniki analityczne do zbiorczego pliku `benchmark_results.csv`.

5.2 Poprawność algorytmiczna (Testy funkcjonalne)

Silnik wyliczający C++ przeszedł systematyczne testy na predefiniowanych, nietypowych i zdegenerowanych scenariuszach geometryczno-logistycznych, które zostały przygotowane specjalnie do weryfikacji warunków brzegowych algorytmów:

- **Układy puste i bezwzględnie izolowane:** Pomyślnie zweryfikowano odporność grafu i zachowanie zmiennych na strukturach pozbawionych kompatybilności (zablokowany przydział krasnoludków ze względu na brak wymaganych minerałów).

- **Złożone formy geometryczne:** Poprawność detekcji punktów w otoczce wypukłej sprawdzano m.in. dla załamanego kształtu "podkowy" oraz rygorystycznie lustrzanych rozstawień osiowych. Testy te potwierdziły, że algorytm skanowania Grahama poprawnie rozpoznaje zwroty na płaszczyźnie, identyfikując wektory kierunkowe i eliminując fałszywe punkty współliniowe, zapewniając zawsze w pełni wypukły wielokąt graniczny.

5.3 Wyniki wydajności (Analiza Benchmarków)

Poniższa tabela prezentuje zagregowane wyniki czasowe dla poszczególnych etapów działania systemu w zależności od skali problemu.

Przypadek testowy	Imp/Graf OK	Czas Imp/Graf	Zrzut OK	Czas Zrzutu	Salwa OK	Czas Salwy
przyklad_bardzo_maly	✓	1.008679 s	✓	0.013850 s	✓	0.014683 s
przyklad_maly	✓	1.008406 s	✓	0.014204 s	✓	0.015379 s
przyklad_sredni	✓	20.015339 s	✓	0.017425 s	✓	0.018775 s
przyklad_duzy	✓	268.095838 s	✓	0.021323 s	✓	0.021891 s
przyklad_mamut	✓	15541.448640 s	✓	0.235619 s	✓	0.035627 s

Wnioski z tabeli wyników:

- **Sieć Grafowa (Min-Cost Max-Flow - kolumna "Czas Imp/Graf"):** Cykliczne wywoływanie procedur relaksacji przy pomocy algorytmu Bellmana-Forda w grafie rezydualnym skutkuje zdecydowanie największym kosztem czasowym. Dla przypadku `przyklad_mamut` obliczenia zajmują większość czasu egzekucji programu, co doskonale odzwierciedla asymptotyczną złożoność pesymistyczną metod opartych na niwelowaniu ujemnych cykli.
- **Algorytmy Geometrii i Struktur Drzewiastych (Graham Scan i RMQ):** Czasy wywoływania i sortowania dla tworzenia otoczki (Czas Zrzutu) oraz proces zapytywania drzewa przedziałowego (Czas Salwy) są relatywnie marginalne. Jak widać w tabeli, nawet dla zestawu "mamut" czas budowy i odpytywania RMQ utrzymuje się na niewiarygodnie niskim poziomie rzędu ~ 0.035 s. Zgodnie z matematycznymi dowodami $\mathcal{O}(K \log K)$ dla budowy patrolu na granicach i logarytmicznym zapytywaniu $\mathcal{O}(\log K)$ RMQ dla Dekametrowców, procesory realizują te etapy niemal natychmiastowo.

5.4 Podsumowanie

Empiryczna ewaluacja systemu i zgromadzony materiał analityczny potwierdzają trafność zaprojektowanej architektury obiektowej. Autorskie oprogramowanie pomyślnie odwzorowuje skomplikowane i zależne od siebie modele teoretyczne (od grafów dwudzielnych po struktury segmentowe), zapewniając optymalny (liniowy) narzut pamięciowy i skalując się zgodnie z teoretycznymi przewidywaniami asymptotycznymi, co jednoznacznie udowadniają testy benchmarkowe na zbiorach rzędu "mamut".